

PA 1: Implementing xv6 User Commands and System Calls

Task 1. (5 pts) Implement the uptime utility

I implemented user/uptime.c, which reports the number of clock ticks since the system began and calls the uptime() system call. The build creates a stub in user/usys.S that traps into the kernel, where sys_uptime() in kernel/sysproc.c returns the global ticks counter. uptime() is declared in user/user.h. In order to include it into the xv6 file system, I added \$U/_uptime to UPROGS in the Makefile.

```
gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ grep -n "scheduler: running" kernel/*.c
kernel/proc.c:453:     printk("scheduler: running pid %d (%s)\n", p->pid, p->name);
gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ nano +453 kernel/proc.c
gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ nano +453 kernel/proc.c
gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ make qemu
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/gchaviraas/xv6-riscv=. -f
march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-bu
iltin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-bu
iltin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -fno-builtin-protect -fno-pie
-no-pie -c -o kernel/proc.o kernel/proc.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/start.o kernel/console.o kernel/printk.o kernel/u
art.o kernel/kalloc.o kernel/spinlock.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/switch.o kernel/trampoline.o kernel/trap.o kernel/s
yscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kern
elvec.o kernel/plic.o kernel/virtio_disk.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if
=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ uptime
up 117 clock ticks
$ uptime
up 171 clock ticks
$
```

Figure 1: Running uptime twice. The tick count increases from 117 to 171 between runs.

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

I implemented user/time1.c, which executes a command and displays the duration in clock ticks. The application prints a usage message and terminates if the user doesn't give a command. If not, it records the start time with uptime() and then calls fork(). The child executes the requested command using `exec(argv[1], &argv[1])`. The right arguments are provided by `&argv[1]` because `argv` is null-terminated. The child prints an error and exits if `exec` fails. After waiting for the child to finish, the parent uses `uptime()` to calculate and print the amount of time that has passed. Additionally, I added `\$U/\_time1` to the Makefile's `UPROGS`.

```
gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if
=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ time1
usage: time1 <command> [args...]
$ time1 sleep 10
elapsed time: 10 ticks
$ time1 asdasd
time1: exec asdasd failed
elapsed time: 0 ticks
$
```

Figure 2: time1 with no arguments prints a usage message, time1 sleep 10 reports about 10 ticks, and a nonexistent command reports an exec error.

```

gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ time1 matmul 10
Time: 50 ticks
elapsed time: 50 ticks
$ time1 matmul 10 & time1 matmul 10 &
$ Time: 100 ticks
elapsed time: 102 ticks
Time: 100 ticks
elapsed time: 101 ticks
$

```

Figure 3: A single `matmul 10` takes 50 ticks. Two concurrent runs take about 101–102 ticks each.

Task 3. (10 pts) Keep track of how much `cputime` a process has used.

I modified three files in the kernel to track CPU time: `kernel/proc.h`, `kernel/proc.c`, and `kernel/trap.c`.

kernel/proc.h: Added `uint cputime`; to struct `proc` so each process tracks the CPU ticks it has used.

kernel/proc.c: In `allocproc()`, I set `p->cputime = 0`; after assigning the PID. This ensures reused process slots start with a fresh counter.

kernel/trap.c: `yield()` is called by the timer interrupt, which happens every tick. In order to give the running process credit for that tick, I increase `cputime` before `yield()`.

In `usertrap()`, I added `p->cputime++`;

In `kerneltrap()`, I added `myproc()->cputime++`; when a process is running.

```

gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ grep -n "cputime" kernel/proc.h kernel/proc.c kernel/trap.c
kernel/proc.h:91:  uint cputime;
kernel/proc.c:127:  p->cputime = 0;
kernel/proc.c:424:// (currently just cputime) to user address raddr, if non-zero.
kernel/proc.c:458:      ru.cputime = pp->cputime;
kernel/trap.c:86:  p->cputime++;
kernel/trap.c:160:  myproc()->cputime++;
gchaviraas@gus-ubuntu-vm:~/xv6-riscv$

```

Figure 4: Output of `grep -n "cputime"` showing the new field in `proc.h`, its initialization in `allocproc()`, and the increments in `usertrap()` and `kerneltrap()`. Lines 424 and 458 of `proc.c` belong to Task 4.

Task 4. (10 pts) Implement a `wait2()` system call that waits for a child to exit and returns the child's status and `rusage`.

`wait2()` works like `wait()`, but also returns the child's CPU time. Adding it required changes to eight files.

kernel/pstat.h: Created struct `rusage` with a `uint cputime` field to store the child's CPU usage.

user/user.h: Added the `rusage` declaration and `wait2()` prototype so user programs can call it.

user/usys.pl: Added `entry("wait2")`; to generate the system call assembly stub.

kernel/syscall.h: Added `SYS_wait2 23` as the system call number.

kernel/syscall.c: Added `sys_wait2` to the system call table so `xv6` can find the correct kernel function.

kernel/sysproc.c: Implemented `sys_wait2()`, using `argaddr()` to get the user-space addresses for the exit status and `rusage`, then passing them to `kwait2()`.

kernel/proc.c: Added kwait2(), based on kwait(). When it finds a zombie child, it copies the exit status and the child's cputime into a struct rusage, then uses copyout() to return it to the parent before freeing the process.

kernel/defs.h: Added the kwait2() prototype so sys_wait2() can call it.

```
gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ grep -n "wait2" kernel/syscall.h kernel/syscall.c kernel/defs.h kernel/sysproc.c kernel/proc.c user/user.h user/usys.pl
kernel/syscall.h:24:#define SYS_wait2 23
kernel/syscall.c:106:extern uint64 sys_wait2(void);
kernel/syscall.c:134: [SYS_wait2] = sys_wait2,
kernel/defs.h:101:int kwait2(uint64, uint64);
kernel/sysproc.c:40:sys_wait2(void)
kernel/sysproc.c:46: return kwait2(p, r);
kernel/proc.c:427:kwait2(uint64 addr, uint64 raddr)
kernel/proc.c:454: // New for wait2: copy the child's rusage to the parent.
user/user.h:10:int wait2(int*, struct rusage*);
user/usys.pl:27:entry("wait2");
gchaviraas@gus-ubuntu-vm:~/xv6-riscv$
```

Figure 5: Every place wait2 appears, showing the complete path from the user-level declaration to the kernel implementation.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

I implemented user/time.c by copying time1.c and making three changes. First, I added `#include "kernel/pstat.h"` so time could use the full struct rusage definition. Second, I replaced `wait()` with `wait2(&status, &ru)` to receive the child's CPU time. Third, I changed the output to show elapsed time, CPU time, and CPU percentage. I also added `$U/_time` to UPROGS in the Makefile. CPU percentage is calculated as $(\text{cpu} * 100) / \text{elapsed}$.

The results matched expectations:

- **No arguments:** Prints a usage message.
- **Single matmul 10:** About 50 elapsed and CPU ticks, or 100% CPU.
- **Two matmul 10 runs:** About 100 elapsed and 50 CPU ticks each, or 50% CPU, since they share one CPU.
- **sleep 10:** 10 elapsed ticks and 0 CPU ticks because the process is sleeping.
- **Nonexistent command:** exec fails, but the child may still receive a CPU tick if a timer interrupt occurs before it exits.

These results confirm that cputime only counts time while a process is running and that wait2() correctly returns the CPU time to the parent.

```

gchaviraas@gus-ubuntu-vm:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ time
usage: time <command> [args...]
$ time matmul 10
Time: 50 ticks
elapsed time: 50 ticks, cpu time: 50 ticks, 100% CPU
$ time matmul 10 & time matmul 10 &
$ Time: 101 ticks
Time: 99 ticks
elapsed time: 100 ticks, cpu time: 50 ticks, 50% CPU
elapsed time: 101 ticks, cpu time: 51 ticks, 50% CPU

$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time asfas
time: exec asfas failed
elapsed time: 1 ticks, cpu time: 1 ticks, 100% CPU
$

```

Figure 6: Output from the time command showing a usage message, a single matmul 10, two concurrent matmul 10 runs, sleep 10, and a nonexistent command.

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

I learned how fork, exec and wait work together to run and measure another program. I learned the difference between elapsed time and CPU time. I also learned how timer interrupts drive scheduling in xv6 and how counting them measures CPU time.

Some of the difficulties I had had to do with github, using linux in general and c syntax. For github it wouldn't let me add, commit and push because of some identity reason, I had to create a token in my github account and use that to push. For linux I struggled mostly with moving around in the terminal, I don't use it regularly so I forget how to move around. Finally, I had to go back and forth to the w3schools and geeksforgeeks website and claude to understand what certain things do and refresh my mind with the syntax.

AI Usage: I used Claude throughout the assignment for guidance, debugging, and explanations. It helped me identify where to make changes in xv6 and how to navigate through a linux terminal, understand some concepts such as system calls, timer interrupts and copyout(). I used Claude to provide the code for uptime.c, time1.c, time.c, and the kernel changes for Tasks 3 and 4 and guide me through the implementation. I then typed the code myself, built and tested it, and asked Claude to explain how each part worked so I could understand the changes I was making. For the report Claude gave me feedback on my spelling and wording.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).